The Syntax of FCMF

Vdash

November 10, 2026

1 Types

Definition 1.1 (Types)**.** The types are defined as follows:

- e, t and u are types.
- If σ and τ are types, so is $(\sigma \to \tau)$.

The types e, t, u are called *base types*, and types of the form $(\sigma \to \tau)$ are called *functional types*. The set of types is written Type.

Convention 1.2 (Has type)**.** We write $\xi : \sigma$ as short for "ξ has type σ".

Convention 1.3 (Bracketing of types)**.** Type brackets associate to the right, so $(\sigma \to (\tau \to \rho))$ is shortened to $(\sigma \to \tau \to \rho)$, and the outermost brackets are omitted. In general we write $\sigma_1 \to \cdots \to \sigma_n \to \tau$ for $(\sigma_1 \to (\sigma_2 \to \ldots (\sigma_n \to \tau) \ldots))$.

For $0 \leq k \leq n$ and a base type τ, call $\sigma_{k+1} \to \cdots \to \sigma_n \to \tau$ the *k-tail* of $\sigma_1 \to \cdots \to \sigma_n \to \tau$. Tails with different k have different numbers of arrows, hence are different types.

2 Symbols

FCMF has *constants* and *variables*.

- The constants are given by the lexicon of FCMF, which is not part of the syntax defined here. Every constant has exactly one type. Constants with the same written form and different types are different constants.
- The *variable names* are exactly

  $$x_1, x_2, \ldots, \qquad \alpha_1, \alpha_2, \ldots, \qquad \beta_1, \beta_2, \ldots, \qquad \gamma_1, \gamma_2, \ldots,$$

  and nothing else is a variable name. The names α_i, β_i, γ_i are *pronoun names* for first-, second- and third-person respectively. A *variable* of type σ is a variable name together with the type σ, written x^σ. A pronoun name has the fixed type e, so there are only the variables α_i^e, β_i^e, γ_i^e. Thus, the set Var_σ of variables of type σ is

  $$\mathsf{Var}_\sigma = \{x_i^\sigma : i \geq 1\}\ (\sigma \neq e), \qquad \mathsf{Var}_e = \{x_i^e, \alpha_i^e, \beta_i^e, \gamma_i^e : i \geq 1\}.$$

  These sets are pairwise disjoint and disjoint from the set of constants; Var is their union. Variables whose name is a pronoun name are called *pronoun variables*; they are never bound by a λ (see Definition 3.3).

Convention 2.1 (Letters for variables)**.** In the object language, we can simply use x for x_1; α for α_1; β for β_1; γ for γ_1 if it causes no ambiguity (In metalanguage, we simply use x to say of any variable). Besides x_i we may use y and z as variable names: y and z are abbreviations of x_2 and x_3 respectively. (For upper and lower case, see Section 5.)

3 Terms

FCMF has no brackets in its terms. Terms are therefore defined directly as strings of *tokens*, and it can be proved that every term can be read in exactly one way.

Definition 3.1 (Tokens)**.** There are three kinds of tokens.

- A *constant token* c^σ, for each constant c of type σ.
- A *variable token* x^σ, for each $x \in \mathsf{Var}_\sigma$.
- A *binder token* $\lambda_\tau x^\sigma.$, for each non-pronoun variable $x \in \mathsf{Var}_\sigma$ and each type τ. Here σ is the type of the bound variable and τ is the type of the scope (see below).

Constant tokens and variable tokens are called *atoms*. A *string* is a finite sequence of tokens; its length is the number of its tokens.

Before we define *terms*, let us first define *head* and *argument*:

Definition 3.2 (Head, argument)**.** Let $M = HN_1 \cdots N_k$ with $k \geq 0$ and H an atom or a λ-abstract (see below). Then H is the *head* of M and $N_1, \ldots, N_k$ are its *arguments*.

Definition 3.3 (Terms)**.** The set L_σ of *terms of type* σ is defined, for all σ simultaneously, as a set of strings.

(T1) Every constant token c^σ and every variable token x^σ is in L_σ.

(T2) If $M \in \mathsf{L}_\tau$ and $\lambda_\tau x^\sigma.$ is a binder token, then the string $\lambda_\tau x^\sigma.M$ is in $\mathsf{L}_{\sigma\to\tau}$. Such a term is a λ-*abstract*, and M is its *scope*.

(T3) Let $H \in \mathsf{L}_\rho$ be a head, where $\rho = \sigma_1 \to \cdots \to \sigma_n \to \tau$ and τ is a base type. If $1 \leq k \leq n$ and $N_i \in \mathsf{L}_{\sigma_i}$ for $1 \leq i \leq k$, then the string $HN_1N_2 \cdots N_k$ is in $\mathsf{L}_{\sigma_{k+1}\to\cdots\to\sigma_n\to\tau}$, i.e. it has the k-tail of ρ as its type.

Only strings that can be built by these rules are terms. The set of all terms is $\mathsf{L} = \bigcup_\sigma \mathsf{L}_\sigma$. A term of type t is a *proposition*, and a term of type u is an *utterance*.

A complete sentence of FCMF is expected to have type u, that is, it is expected to be an utterance. Additionally, when we speak of heads or arguments of M we usually assume M is a term of FCMF if not otherwise indicated.

Every term has the form $HN_1 \cdots N_k$ with H a head and $k \geq 0$. Note that no brackets occur: the type τ written in a binder token marks where the scope of the λ ends, and the types of heads mark how many arguments they can at most take.

This definition allows no ambiguity in reading despite we do not have brackets:

Proposition 3.4 (Unique Reading)**.** Every term M has exactly one type, and there are unique H, $k \geq 0$ and $N_1, \ldots, N_k$ such that $M = HN_1 \cdots N_k$ where H is an atom or a λ-abstract, and the conditions of (T1)–(T3) hold.

The proof is left for the appendix.

However, for human users it may be sometimes difficult to check what the scope of a λ-abstract exactly is. Thus we introduce the following convention:

Convention 3.5 (Semicolon)**.** If $\lambda_\sigma x^\sigma.M$ is a lambda abstract, one may write $\lambda_\sigma x^\sigma.M$; in the place of $\lambda_\sigma x^\sigma.M$.

But it should be noted that semicolons are not a part of the term of FCMF.

Finally, we define free and bound occurrences of variables and the distinction between open and closed terms:

Definition 3.6 (Free variable)**.** The set $\mathsf{FV}(M)$ of free variables of a term M is defined inductively:

- $\mathsf{FV}(c^\sigma) = \varnothing$ for a constant token c^σ;
- $\mathsf{FV}(x^\sigma) = \{x^\sigma\}$ for a variable token x^σ;
- $\mathsf{FV}(HN_1 \cdots N_k) = \mathsf{FV}(M) \cup \mathsf{FV}(N_1) \cup \cdots \cup \mathsf{FV}(N_k)$;
- $\mathsf{FV}(\lambda_\tau x^\sigma.M) = \mathsf{FV}(M) \setminus \{x^\sigma\}$.

A term M is *closed* if $\mathsf{FV}(M) = \varnothing$, and *open* otherwise.

4 Type superscripts

Convention 4.1 (Type annotation)**.** Every token carries its type: a constant token c^σ and a variable token x^σ carry the type σ, and a binder token $\lambda_\tau x^\sigma.$ carries both the type σ of the bound variable and the type τ of its scope. The type of a complex term is determined by those of its tokens and is never written. Thus, in written FCMF every constant and every variable carries its type, and every λ carries the type of its scope as a subscript, apart from the following exceptions.

Convention 4.2 (Exceptions)**.** (E1) *Subscripts* For some constants such as $\exists$, $\forall$, $=$, etc., the type is indicated by a *subscript* instead of a superscript, giving the type σ of the things they receive:

$$\exists_\sigma,\ \forall_\sigma \ \text{ stand for } \ \exists^{(\sigma\to t)\to t},\ \forall^{(\sigma\to t)\to t}, \qquad =_\sigma \ \text{ stands for } \ =^{\sigma\to\sigma\to t}.$$

No further superscript is written on these constants.

(E2) *Bound variables.* The type of a bound variable should be written in its binder token: $\lambda_\tau x^\sigma.$. Other occurrences of x in the scope of this binder token may be written without superscript if it does not cause ambiguity, and have the type of the nearest enclosing binder token that binds that variable. A free variable always carries its superscript.

(E3) *Pronoun variables.* Since α_i, β_i, γ_i have the fixed type e, they may be written without superscript.

5 Orthographic case

A written form of a term is *alphabetic* if it comprises Latin letters, and *caseless* otherwise (e.g. $\wedge$, $\exists$, $\vdash$, α_i).

Definition 5.1 (Case)**.** Alphabetic written forms that differ only in the capitalization denote the same symbol. Case is not part of the identity of a symbol; it is a variant of notation.

Case rule. Every occurrence of an alphabetic symbol, if it is not a part of a binder token, is written with a capitalized letter (*head form*) iff it is a head of a subterm $HN_1 \cdots N_k$ with $k \geq 1$, and with a lower case (*argument form*) otherwise. Variables in a binder token is always written with a lower case. Caseless symbols are written in the same way in both positions.

6 Examples

Example 6.1. "Socrates has a good property."

$$\vdash^{t\to u} \exists_{e\to t}\lambda_t x^{e\to t}.\ \wedge^{t\to t\to t} X\mathrm{socrates}^e\mathrm{GOOD}^{(e\to t)\to t}x$$

We may check that this string has type u as follows:

$$\dfrac{\dfrac{\wedge : t \to t \to t \qquad \dfrac{x : e \to t \qquad \text{socrates} : e}{X\text{socrates} : t}\,\text{App}}{\wedge X\text{socrates} : t \to t}\,\text{App} \qquad \dfrac{\text{GOOD} : (e \to t) \to t \qquad x : e \to t}{\text{GOOD}x : t}\,\text{App}}{\dfrac{\wedge X\text{socrates}\,\text{GOOD}x : t}{\lambda_t x^{e\to t}.\wedge X\text{socrates}\,\text{GOOD}x : (e \to t) \to t}\,\text{Abs}}\,\text{App}$$

$$\text{Let } \Phi := \lambda_t x^{e\to t}.\wedge^{t\to t\to t} X\text{socrates}^{e}\,\text{GOOD}^{(e\to t)\to t}x$$

$$\dfrac{\vdash : t \to u \qquad \dfrac{\exists_{e\to t} : ((e \to t) \to t) \to t \qquad \Phi : (e \to t) \to t}{\exists_{e\to t}\,\Phi : t}\,\text{App}}{\vdash \exists_{e\to t}\,\Phi : u}\,\text{App}$$

Example 6.2. "I like him/her/it."

$$\vdash^{t\to u} \text{SIS}^{e\to e\to t}\alpha\gamma$$

Here α and γ are pronoun variables of type e and carry no superscript, by (E3).

Example 6.3. "That predicate is good." A demonstrative of type $e \to t$ is not a pronoun variable, so it is an ordinary free variable that carries its type:

$$\vdash^{t\to u} \text{GOOD}^{(e\to t)\to t}y^{e\to t}$$

Example 6.4. "The cat is on a mat." It is always possible to follow Russell in definite descriptions:

$$\vdash^{t\to u} \exists_e\lambda_t x^e.\wedge^{t\to t\to t}\exists_e\lambda_t y^e.\wedge^{t\to t\to t}\wedge^{t\to t\to t}\text{BIL}^{e\to t}y\,\forall_e\lambda_t z^e.\supset^{t\to t\to t}\text{BIL}^{e\to t}z =_e zy\,\text{AM}^{e\to e\to t}xy\,\text{TIKIR}^{e\to t}x$$

which can be abbreviated into:

$$\vdash^{t\to u} \exists_e\lambda_t x^e.\wedge^{t\to t\to t}\text{AM}^{e\to e\to t}x\iota_e\text{bil}^{e\to t}\text{TIKIR}^{e\to t}x$$

(ι_σ has type $(\sigma \to t) \to \sigma$)

This is, strictly speaking, false even if there is a particular cat on a mat, since there are more than one cats in the world. In natural language, in these case we consider the quantifier to be implicitly restricted. This is a pragmatic phenomenon, thus FCMF of course can be used such way. However, another way to translate the natural language sentence "The cat is on a mat" is using demonstratives — that is, free variables:

$$\vdash^{t\to u} \exists_e\lambda_t x^e.\wedge^{t\to t\to t}\text{AM}^{e\to e\to t}xy^e\wedge^{t\to t\to t}\text{BIL}^{e\to t}y^e\text{TIKIR}^{e\to t}x$$

Or one may use the third-person pronoun if one wishes:

$$\vdash^{t\to u} \exists_e\lambda_t x^e.\wedge^{t\to t\to t}\text{AM}^{e\to e\to t}x\gamma\wedge^{t\to t\to t}\text{BIL}^{e\to t}\gamma\text{TIKIR}^{e\to t}x$$

Example 6.5. "What is a relation between Socrates and Plato?"

$$\text{WHAT?}^{((e\to e\to t)\to t)\to u}\lambda_t x^{e\to e\to t}.X\text{socrates}^{e}\text{plato}^{e}$$

Example 6.6. "The relation between Plato and Aristotle is not transitive"

$$\vdash^{t\to u} \lambda_t x^{e\to e\to t}.\neg^{t\to t}\forall_e\lambda_t y^e.\forall_e\lambda_t z^e.\forall_e\lambda_t x_4^e.\supset^{t\to t\to t}\wedge^{t\to t\to t}Xyz Xzx_4 Xyx_4; \iota_{e\to e\to t}\lambda_t x^{e\to e\to t}.X\text{plato}^{e}\text{aristotle}^{e}$$

Again, literally taken this is false because there are more than one relation between Plato and Aristotle. The above sentence is lengthy, considering that ι is already an abbreviation. One may introduce the predicate "$\text{TRIRE}^{(e\to e\to t)\to t}$", and write:

$$\vdash^{t\to u} \neg^{t\to t}\text{TRIRE}^{(e\to e\to t)\to t}\iota_{e\to e\to t}\lambda_t x^{e\to e\to t}.X\text{plato}^{e}\text{aristotle}^{e}$$

And if one uses demonstratives:

$$\vdash^{t\to u} \lambda_t x^{e\to e\to t}.\wedge^{t\to t\to t}X^{e\to e\to t}\text{plato}^{e}\text{aristotle}^{e}\neg^{t\to t}\text{TRIRE}^{(e\to e\to t)\to t}x^{e\to e\to t}; y^{e\to e\to t}$$

Readers may check that every example above has type u.

A The proof of Proposition 3.4

We first prove the prefix-free lemma:

Lemma A.1 (Prefix-freeness)**.** Let σ be a type, $w \in \mathsf{L}_\sigma$, and let u be a string such that $wu \in \mathsf{L}_\sigma$. Then u is empty.

Proof of Lemma A.1. By strong induction on the length of wu. Every term has the form $HN_1 \cdots N_k$ ($k \geq 0$) with H an atom or a λ-abstract $\lambda_\tau x^{\sigma'}.P$ with $P \in \mathsf{L}_\tau$ (by (T1)–(T3); (T2) is the case $k = 0$). Write

$$w = HN_1 \cdots N_k, \qquad wu = H'N'_1 \cdots N'_{k'}.$$

Step 1: $H = H'$. Both begin with the same first token. If it is an atom, then $H = H'$. If it is a binder token $\lambda_\tau x^{\sigma'}.$, then $H = \lambda_\tau x^{\sigma'}.P$ and $H' = \lambda_\tau x^{\sigma'}.P'$ with $P, P' \in \mathsf{L}_\tau$. Both are initial segments of the string following the binder token, so one is a prefix of the other. Both are shorter than wu, so by the induction hypothesis $P = P'$.

Step 2: $k = k'$. Let $\rho = \sigma_1 \to \cdots \to \sigma_n \to \beta$ (β base) be the type of H; it is determined by the first token(s) of H. By (T3), w has the k-tail of ρ and wu has the k'-tail of ρ as type. Both equal σ, and the k-tail has $n - k$ arrows, so $n - k = n - k'$, i.e. $k = k'$.

Step 3: $N_i = N'_i$ for all i. Suppose $N_j = N'_j$ for $j < i$. Then N_i and N'_i both start at the same position and both are in L_{σ_i}, so one is a prefix of the other. Both are shorter than wu, so the induction hypothesis gives $N_i = N'_i$.

Hence w and wu are the same string and u is empty. □

Proof of Proposition 3.4. Existence is immediate from the definition of L: M is built by (T1), (T2) or (T3), which gives the form $HN_1 \cdots N_k$ with $k = 0, 0, \geq 1$ respectively.

Uniqueness. Let $M = HN_1 \cdots N_k = H'N'_1 \cdots N'_{k'}$ be two readings. As in Step 1 of the Lemma, $H = H'$ (for an abstract head, $P = P'$ by Lemma A.1 applied to the types τ given by the common binder token). Hence H and H' have the same type $\rho = \sigma_1 \to \cdots \to \sigma_n \to \beta$. Suppose $N_j = N'_j$ for all $j < i$ with $i \leq \min(k, k')$. Then $N_i, N'_i \in \mathsf{L}_{\sigma_i}$ start at the same position, so by Lemma A.1 $N_i = N'_i$. If $k < k'$, then after $N_k = N'_k$ the string M has ended, but N'_{k+1} is a nonempty string occurring after it; contradiction. The case $k' < k$ is symmetric. So $k = k'$ and the two readings coincide. Finally, the type of M is the k-tail of ρ, determined by H and k, so M has exactly one type. □